

Bandit - Level 12

Goal

The password for the next level is stored in `data.txt`, which is a **hexdump of a file that has been repeatedly compressed**. The level suggests creating a working directory under `/tmp` with `mkdir` or `mktemp -d`, copying the file with `cp`, and renaming it with `mv`.

Connection

```
ssh -p 2220 bandit12@bandit.labs.overthewire.org
```

Use the password retrieved at the end of [Bandit - Level 11](#).

Concepts

The file is a **nested archive** — nine layers of compression applied in sequence, then converted to a hexdump. Unwrapping means removing one layer at a time, outermost first.

The hexdump, and why `xxd` comes first

A **hexdump** is a **text representation of binary data**. Each byte becomes two hexadecimal characters, laid out in three columns:

```
00000000: 1f8b 0808 a6f0 3b6a 0203 6461 7461 322e  ....;j..data2.
|           |
|           |  the bytes, in hex, grouped in pairs  |  the same bytes as ASCII,
|           |                                     |  with '.' where the byte
|  byte offset from the start of the file,         |  isn't printable
|  in hex (00000010 = byte 16)
```

That layout explains the file's odd behaviour: `cat data.txt` printed cleanly and `file` called it ASCII text, because **it genuinely is text**. What it isn't is the actual data — it's the data *written out in hexadecimal*, which roughly doubles the size and makes every byte safely printable.

This is the same problem base64 solves in [Bandit - Level 10](#) — moving binary through a text-only channel — with the opposite trade-off. Base64 is compact (+33%) but unreadable; a hexdump is bulky (+100%) but human-inspectable. Hexdumps are used where a person needs to see the bytes; base64 where a machine just needs to transport them.

```
xxd file          # produce a hexdump
xxd -r dump > file # revert a hexdump back to bytes
```

`-r` is the whole first step: it reads the hex columns, ignores the offset and ASCII columns, and reconstructs the original bytes.

Useful flags beyond this level:

Flag	Effect
<code>-l N</code>	dump only the first N bytes
<code>-s N</code>	start at offset N (<code>-s 0x100</code> accepts hex)
<code>-c N</code>	N bytes per line (default 16)
<code>-g N</code>	group bytes in blocks of N (default 2)
<code>-p</code>	plain hex, no offsets or ASCII columns
<code>-b</code>	binary digits instead of hex
<code>-i</code>	output as a C array — used to embed a file in source code

The readability is not incidental. Before running a single command, `head -3 hexdump.txt` already revealed `1f8b` at offset 0 (gzip's magic number) and the string `data2.bin` in the ASCII column — the format and the original filename, read straight off the page. `hexdump -C` produces near-identical output and is the more commonly installed alternative; `od -A x -t x1z` is the POSIX fallback when neither is present.

Each layer uses a different method, and the next layer is unknown until the current one is removed. That forces a loop rather than a fixed sequence of commands:

```
file X → identify → rename with the required extension → decompress → file X
→ ...
```

Three tools cover every layer:

file reports	extension	command	consumes input?
gzip compressed data	<code>.gz</code>	<code>gunzip</code>	yes
bzip2 compressed data	<code>.bz2</code>	<code>bunzip2</code>	yes
POSIX tar archive	<code>.tar</code>	<code>tar -xf</code>	no
ASCII text	—	<code>cat</code>	—

Why the renaming is needed. `gunzip` refuses to run on a file without a `.gz` suffix (`unknown suffix - ignored`) — a safety check against clobbering a file that isn't what the user assumed. `bunzip2` is more permissive: it decompresses anyway and warns `Can't guess original name` , inventing a `.out` suffix. The `mv` exists to satisfy that check, not because the tools need the extension to work.

Both accept `-c` (`--stdout`), which writes to stdout and **skips the extension check entirely** — the clean form once the behaviour is understood:

```
gunzip -c input > output
bunzip2 -c input > output
```

`tar` **behaves differently in two ways** that break the rhythm of the loop: it does not delete its input, and it extracts under the name stored *inside* the archive (`data5.bin` , `data8.bin` ...), not the name given to it. After every `tar -xf` , `ls` is mandatory — the output filename cannot be predicted.

tar packages, it does not compress. `gzip` and `bzip2` compress a single stream with no concept of filenames or permissions; `tar` bundles multiple files with their metadata into one stream without compressing. `.tar.gz` is both operations chained. This level shows them separated.

Attempts

```
mktemp -d david
# → mktemp: too few X's in template 'david'
# mktemp generates a RANDOM name from a template; it needs at least 3 X's
# to substitute. Correct form: mktemp -d /tmp/davidXXXXXXXXX

cd..          # → cd.: command not found (missing space)
cd /temp      # → No such file or directory (it's /tmp)

cd /tmp && mkdir david && ls
# → ls: cannot open directory '.': Permission denied
# mkdir SUCCEEDED – the error came from ls. Bandit's /tmp has read access
# removed so players can't enumerate each other's directories. Directory
# permissions: r = list names, x = enter and access by name. Without r but
# with x, you can work inside it but not see what's there.
```

The destructive mistake:

```
mv data.txt bandit12.txt          # bandit12.txt is now the only file with data
xxd -r data.txt > bandit12.txt    # reads a name that no longer exists,
                                   # and writes over the file that HAS the data

# → xxd: data.txt: No such file or directory
# Result: command failed AND the file was emptied to 0 bytes.
```

The shell **processes redirections before running the command:** `> bandit12.txt` opened and truncated the file to zero, and only then did `xxd` run and fail. One line, two losses.

```
cat stage1          # on gzip data – dumped raw bytes to the terminal
# Control bytes corrupted the input buffer; subsequent commands appeared as
# "2mv" and "/mv". Fixed with `reset`. Same reason grep refuses binary input
# in [[Bandit – Level 09]].

mv data5.bin next.gz && gunzip next
# → gzip: next.gz: not in gzip format
# file had reported POSIX tar archive – .gz was the wrong extension.
# gzip read the magic bytes and refused. The rename did not fool it.

bunzip2 next2
# → bunzip2: Can't guess original name for next2 -- using next2.out
# NOT an error. It decompressed successfully and invented an output name.

tar -xf next2.tar    # → Cannot open: No such file or directory
# The file was next2.out. Assumed the name instead of reading it from ls.
```

Solution

Working directory (hard-to-guess name, per the level text — `/tmp` is shared):

```
mkdir -p /tmp/dk9x2mqf7t && cd /tmp/dk9x2mqf7t
cp ~/data.txt .
mv data.txt hexdump.txt
```

Inspect before acting:

```
head -3 hexdump.txt
# 00000000: 1f8b 0808 a6f0 3b6a 0203 6461 7461 322e  ....;j..data2.
# 00000010: 6269 6e00 0144 02bb fd42 5a68 3931 4159  bin..D...BZh91AY
```

Offset 0 already answers the first question: `1f 8b` is gzip's magic number, and the ASCII column shows the stored original name `data2.bin`. `BZh` on the next line is bzip2's signature — layer two visible before layer one is opened.

Reverse the hexdump, then loop:

```
xxd -r hexdump.txt > stage1
file stage1      # gzip compressed data, was "data2.bin"

mv stage1 stage1.gz && gunzip stage1.gz
file stage1      # bzip2 compressed data

mv stage1 stage1.bz2 && bunzip2 stage1.bz2
file stage1      # gzip compressed data, was "data4.bin"

mv stage1 stage1.gz && gunzip stage1.gz
file stage1      # POSIX tar archive (GNU)

mv stage1 stage1.tar && tar -xf stage1.tar && ls
file data5.bin   # POSIX tar archive (GNU)

mv data5.bin stage2.tar && tar -xf stage2.tar && ls
file data6.bin   # bzip2 compressed data

bunzip2 -c data6.bin > stage3
file stage3      # POSIX tar archive (GNU)

mv stage3 stage3.tar && tar -xf stage3.tar && ls
file data8.bin   # gzip compressed data, was "data9.bin", original size 49

mv data8.bin data8.gz && gunzip data8.gz
file data8       # ASCII text

cat data8
# → The password is qQYQiHOBPR8zR61qxYqX45quvihF2uzk
```

original size modulo 2^{32} 49 on the final layer is the tell that it's the last one — 49 bytes is exactly the length of the password line.

Points of Friction

1. `cmd input > output` where input and output are the same file destroys it. Redirections are resolved before the command runs, so the target is truncated to zero before anything reads it. The rule: **source and destination of a redirect must never be the same file**, directly or after a rename. To transform a file in place, write to a temporary name and then `mv`.

2. **Renaming and then reading the old name.** The `mv` suggested by the level text is for the *later* layers, where `gunzip` demands a suffix. Renaming is not the problem; losing track of which name currently holds the data is.

3. `cat` on a binary corrupted the terminal. `file` had already reported gzip data. Safe inspection: `head -c 32 file | xxd`, which was used correctly later. Recovery: `reset`.

4. **Writing an extension before reading** `file`. Applying `.gz` to a tar archive earned `not in gzip format`. Instructive rather than costly: **the tool reads the magic bytes, not the name**. Renaming a file changes nothing about what it is — the point already recorded in [Linux - File Type Detection](#) since Level 4, demonstrated here nine times over.

5. **Assuming output filenames.** `tar` extracts under its internal name; `bunzip2` invents `.out` when it can't strip a suffix. Neither is predictable. `ls` after every step is the only reliable way to know what appeared — or `ls -lt` to sort by time when old `.tar` files accumulate.

6. `mktemp -d david` **needs X's**. `mktemp` generates a *random* name from a template and requires at least three `X` characters to substitute: `mktemp -d /tmp/davidXXXXXXXX`. It prints the created path to stdout.

7. **Read Permission denied from `ls` as `mkdir` having failed.** Two different commands, two different operations. On a directory, `r` grants listing and `x` grants traversal — Bandit's `/tmp` has `r` removed for others, so a directory can be created and used but not enumerated. See [Linux - Permissions & Process Management](#).

Key Takeaway

The level is nine repetitions of one idea: **a filename tells you nothing about a file's contents; only its bytes do**. Every layer had to be identified with `file` because the name was either absent, wrong, or inherited from an unrelated archive — and when a wrong extension was applied deliberately, the tool caught it instantly by reading the magic number.

The hexdump makes the same point from the other direction: a file can be **perfectly valid text and still not be the data**. `cat` worked, `file` said ASCII — and both were true and useless. `xxd -r` was needed to get at what the text was describing. Recognising a hexdump on sight (offset column, hex pairs, ASCII gutter) is worth having, because it shows up in packet captures, debugger output, forensic tooling and vendor bug reports.

The second lesson is structural. The task cannot be solved by a fixed sequence of commands, because each step's output determines the next step's tool. That shape — *inspect, branch, act, re-inspect* — is what makes it automatable as a loop, and is the same shape as unpacking an unknown sample in malware triage. Full treatment: [Linux - Nested Archives and Compression Layers](#). Scripted versions: [Script - Bandit 12 Decompression Loop](#).

Next

```
ssh -p 2220 bandit13@bandit.labs.overthewire.org
```

◀ Previous: [Bandit - Level 11](#) · Next ▶ [Bandit - Level 13](#)